

Resolucion del Practico 3

Testing de Software 2024

Índice

Introduccion	3
1. Ejercicio 1: Lectura del Capitulo 6	3
2. Ejercicio 2: number0f0curreneces – PWC	4
2.1. Archivos trabajados	4
2.2. Modelo del Dominio de Entradas (MDE)	4
2.3. Requisitos de test (PWC)	5
2.4. Casos de prueba	5
2.5. Implementacion	6
2.6. Ejecucion	7
3. Ejercicio 3: intersection – analisis del MDE y BCC	7
3.1. Archivos trabajados	7
3.2. a) Completitud de C1 (analogamente C2)	7
3.3. b) No solapamiento de C1 (analogamente C2)	7
3.4. c) Completitud de C3	7
3.5. d) Disjointness de C3	8
3.6. e) Revision del MDE	8
3.7. f) Base Choice Coverage	8
3.8. Casos implementados	9
3.9. Implementacion	9
3.10. Ejecucion	10
4. Ejercicio 4: PatternIndex – PWC y deteccion de fallas	10
4.1. Archivos trabajados	10
4.2. Modelo del Dominio de Entradas	10
4.3. Requisitos PWC y casos	11

4.4.	Fallas detectadas en la implementacion original	11
4.5.	Correccion aplicada	12
4.6.	Ejecucion	13
5.	Ejercicio 5: Iterator sobre ArrayList – PWC	13
5.1.	Archivos trabajados	13
5.2.	Especificacion de referencia	13
5.3.	MDE	13
5.4.	Requisitos y casos	14
5.5.	Resultado	14
5.6.	Ejecucion	14
	Conclusiones	15

Introduccion

Este documento resuelve el *Practico 3* de Testing de Software. El eje del practico es el **Particionado del Espacio de Entradas** (*Input Space Partitioning*, ISP) y los criterios de cobertura combinatoria asociados (*Each Choice*, *Pair-Wise*, *Base Choice*, *All Combinations*). Toda la resolucioin sigue el flujo recomendado por Ammann y Offutt:

1. identificar la unidad a testear y listar sus parametros,
2. construir un *Modelo del Dominio de Entradas* (MDE) a partir de la especificacion,
3. declarar restricciones de factibilidad,
4. aplicar un criterio para generar requisitos de test,
5. instanciar valores concretos para satisfacer cada requisito.

Cada ejercicio se acompaña del MDE elegido, los requisitos de test (TR) derivados, los casos de prueba que los cubren y, cuando corresponde, la implementacion que satisface dichos casos.

1. Ejercicio 1: Lectura del Capitulo 6

El ejercicio pide leer el Capitulo 6 (*Input Space Partitioning*) del libro *Introduction to Software Testing* (Ammann y Offutt, 2da edicion). Los puntos centrales que estructuran la lectura y que aparecen aplicados en los ejercicios siguientes son:

- **Particionado del dominio.** Toda entrada valida debe caer en algun bloque (*completeness*) y solo en uno (*disjointness*). Si un valor cae en mas de un bloque o en ninguno, la particion esta mal definida y deben revisarse las caracteristicas.
- **Caracteristicas y bloques.** Una *caracteristica* es un escenario relevante para testing (por ejemplo, “la lista esta vacia”). Cada caracteristica induce una particion del dominio en *bloques* (los posibles “modos” de esa propiedad).
- **Dos enfoques para construir el MDE.**
 - **Basado en interfaz:** una caracteristica por parametro, mirando la sintaxis. Es rapido pero ignora relaciones entre parametros.
 - **Basado en funcionalidad:** las caracteristicas se derivan del comportamiento esperado (precondiciones, poscondiciones, valores especiales). Es mas costoso pero mucho mas expresivo.
- **Criterios de combinacion.**

- *Each Choice Coverage (ECC)*: cada bloque aparece al menos en un test.
 - *Pair-Wise Coverage (PWC)*: todo par de bloques de características distintas aparece en algún test.
 - *Base Choice Coverage (BCC)*: se fija un test base y se varía una característica a la vez.
 - *All Combinations (ACoC)*: producto cartesiano de todos los bloques.
- **Restricciones e infeasibility.** No todas las combinaciones son posibles. En ACoC y PWC se descartan las combinaciones inválidas; en BCC se reemplaza un valor por uno no base hasta obtener una combinación factible.
 - **Costo/beneficio.** PWC suele ser un buen compromiso entre tamaño de la suite y capacidad de detección, ya que muchas fallas se disparan por interacción de a dos factores.

Estos conceptos son la base directa de los ejercicios 2 a 5.

2. Ejercicio 2: `numberOfCurrences` – PWC

Se diseña una batería de tests para `ListUtils.numberOfCurrences(List<Integer>l, Integer element)` aplicando *Pair-Wise Coverage* sobre un MDE basado en funcionalidad.

2.1. Archivos trabajados

- `src/main/java/assignment3_exercises/ListUtils.java`
- `src/test/java/assignment3_exercises/numberOfCurrencesTest.java`

2.2. Modelo del Dominio de Entradas (MDE)

Se identifican cuatro características relevantes a partir de la especificación: dos verifican que los parámetros sean no nulos (precondición explícita) y otras dos describen el comportamiento funcional sobre la lista.

- **C1 – referencia de l:**
 - L0: `l == null`
 - L1: `l != null`
- **C2 – referencia de element:**
 - E0: `element == null`

- E1: `element != null`
- **C3 – tamaño de la lista** (solo aplica si L1):
 - S0: lista vacia
 - S1: lista no vacia
- **C4 – cantidad de ocurrencias** (solo aplica si L1 y E1):
 - O0: 0 ocurrencias
 - O1: 1 ocurrencia
 - O2: mas de 1 ocurrencia

Restricciones de factibilidad:

- Si L0 entonces C3 y C4 no aplican: la lista nula dispara excepcion antes de inspeccionar cualquier otra propiedad.
- Si E0 entonces C4 no aplica: sin elemento valido no tiene sentido hablar de ocurrencias.
- Si S0 entonces necesariamente O0: una lista vacia no puede contener al elemento.

2.3. Requisitos de test (PWC)

PWC obliga a que toda pareja de bloques entre características distintas aparezca en al menos un test. Aplicando las restricciones anteriores, los pares factibles son:

- R01: (L0, E0) R02: (L0, E1)
- R03: (L1, E0) R04: (L1, E1)
- R05: (L1, S0) R06: (L1, S1)
- R07: (L1, O0) R08: (L1, O1) R09: (L1, O2)
- R10: (E0, S0) R11: (E0, S1)
- R12: (E1, S0) R13: (E1, S1)
- R14: (E1, O0) R15: (E1, O1) R16: (E1, O2)
- R17: (S0, O0) R18: (S1, O0) R19: (S1, O1) R20: (S1, O2)

2.4. Casos de prueba

TC	Entrada 1	element	Salida	Cubre
TC1	null	null	IAE	R01
TC2	null	7	IAE	R02
TC3	[]	null	IAE	R03, R05, R10
TC4	[1,2,3]	null	IAE	R03, R06, R11
TC5	[]	5	0	R04, R05, R07, R12, R14, R17
TC6	[1,2,3]	9	0	R04, R06, R07, R13, R14, R18
TC7	[1,2,3]	2	1	R04, R06, R08, R13, R15, R19
TC8	[4,1,4,4]	4	3	R04, R06, R09, R13, R16, R20

IAE indica `IllegalArgumentException`. Adicionalmente se incluye un caso de robustez (TC9) que verifica que la rutina no muta la lista de entrada: tras invocar `numberOfOccurrences` sobre `[4,1,4,4]` buscando 4, la lista debe seguir siendo `[4,1,4,4]`.

2.5. Implementacion

```
public static int numberOfOccurrences(List<Integer> l, Integer
    element) {
    if (l == null || element == null) {
        throw new IllegalArgumentException("List and element must
            be non-null");
    }
    int count = 0;
    for (Integer current : l) {
        if (element.equals(current)) {
            count++;
        }
    }
    return count;
}
```

La rutina primero valida las precondiciones (R01–R03 dependen de esto) y luego recorre la lista contando con `element.equals(current)` para que `Integer` se compare por valor y no por referencia.

2.6. Ejecucion

```
mvn -Dmaven.repo.local=.m2 -Dtest=numberOfOccurrencesTest test
```

3. Ejercicio 3: intersection – analisis del MDE y BCC

El ejercicio analiza el MDE original propuesto en el enunciado para `SetUtils.intersection(Set<Integer>set1, Set<Integer>set2)`, corrige sus problemas y genera una suite por *Base Choice Coverage*.

3.1. Archivos trabajados

- `src/main/java/assignment3_exercises/SetUtils.java`
- `src/test/java/assignment3_exercises/IntersectionTest.java`

3.2. a) Completitud de C1 (analogamente C2)

Si, C1 es completa. Para un `Set<Integer>set1` solo hay tres posibilidades mutuamente excluyentes:

- `set1 == null`,
- `set1 != null` y vacio,
- `set1 != null` con al menos un elemento.

Los bloques b1, b2, b3 del enunciado cubren exactamente esas tres posibilidades, por lo que todo `Set<Integer>` concebible cae en algun bloque.

3.3. b) No solapamiento de C1 (analogamente C2)

Si, C1 no solapa. Las tres situaciones son incompatibles entre si: un set nulo no puede ser vacio o no vacio (ni siquiera tiene metodos invocables), y un set vacio no puede ser simultaneamente no vacio. No existe `set1` que caiga a la vez en dos bloques.

3.4. c) Completitud de C3

No, C3 no es completa. Considerese el contraejemplo:

$$\text{set1} = \{1, 2\}, \quad \text{set2} = \{2, 3\}.$$

Aquí hay intersección no vacía ($\{2\}$), pero **set1** y **set2** no son iguales, ninguno es subconjunto del otro y tampoco son disjuntos. Esta entrada no encaja en ninguno de los bloques **b1–b4** originales: existe un *solapamiento parcial* que el MDE no contempla.

3.5. d) Disjointness de C3

No, **C3** no es disjunta. Si “subconjunto” se interpreta de forma no estricta, el caso

$$\text{set1} = \text{set2} = \{1, 2\}$$

cae simultáneamente en tres bloques: representan el mismo conjunto (**b1**), **set1** es subconjunto de **set2** (**b2**) y **set2** es subconjunto de **set1** (**b3**). Hay solapamiento entre los bloques.

3.6. e) Revision del MDE

Para que **C3** sea completa y disjunta se utiliza la noción de subconjunto *propio* y se añade un bloque adicional que cubra el solapamiento parcial:

- **C1 (validez de set1):**
 - C1.b1: `set1 == null`
 - C1.b2: `set1 == {}`
 - C1.b3: `set1` no vacío
- **C2 (validez de set2):** análogo a C1.
- **C3 (relación entre set1 y set2)**, solo aplicable si ninguno es nulo:
 - C3.b1: `set1 = set2`
 - C3.b2: `set1` es subconjunto *propio* de `set2`
 - C3.b3: `set2` es subconjunto *propio* de `set1`
 - C3.b4: `set1` y `set2` disjuntos
 - C3.b5: solapamiento parcial (intersección no vacía y ninguno es subconjunto del otro)

Con estos ajustes **C3** queda completa (todo par (**set1**, **set2**) con ambos no nulos encaja en alguno de los cinco bloques) y disjunta (los cinco bloques se excluyen mutuamente).

3.7. f) Base Choice Coverage

Bloques base elegidos: el caso “camino feliz” es ambos sets no vacíos e iguales.

- C1 base: C1.b3 (`set1` no vacío).
- C2 base: C2.b3 (`set2` no vacío).
- C3 base: C3.b1 (`set1 = set2`).

Requisitos BC (el test base mas un test por cada bloque no base, ajustando factibilidad):

TR	Variacion respecto del base	Resultado esperado
TR-BC1	— (caso base)	{1,2,3}
TR-BC2	C1 → C1.b1 (<code>null</code>)	<code>NullPointerException</code>
TR-BC3	C1 → C1.b2 (<code>{}</code>)	<code>{}</code>
TR-BC4	C2 → C2.b1 (<code>null</code>)	<code>NullPointerException</code>
TR-BC5	C2 → C2.b2 (<code>{}</code>)	<code>{}</code>
TR-BC6	C3 → C3.b2 (subc. propio)	<code>set1</code>
TR-BC7	C3 → C3.b3 (superc. propio)	<code>set2</code>
TR-BC8	C3 → C3.b4 (disjuntos)	<code>{}</code>
TR-BC9	C3 → C3.b5 (solap. parcial)	elementos comunes

Notar que TR-BC3 y TR-BC5 son los casos donde una variacion sobre C1 o C2 vuelve no aplicable a C3; segun la guia de manejo de restricciones para BCC, se sustituye la base de C3 por una combinacion factible (en este caso, interseccion vacia trivial).

3.8. Casos implementados

Cada TC1–TC9 cubre uno de los TR-BCi correspondientes. Adicionalmente la suite incluye:

- Verificacion de no mutacion: `set1` y `set2` deben quedar igual a su copia pre-llamada.
- Verificacion de nueva instancia: el resultado no es `== set1` ni `== set2`.
- Borde adicional: ambos sets vacios → resultado vacio.

3.9. Implementacion

```
public static Set<Integer> intersection(Set<Integer> set1, Set<
    Integer> set2) {
    Objects.requireNonNull(set1, "set1 must be non-null");
    Objects.requireNonNull(set2, "set2 must be non-null");

    Set<Integer> result = new HashSet<>(set1);
    result.retainAll(set2);
    return result;
}
```

```
}
```

La validacion por `Objects.requireNonNull` respeta la especificacion de `NullPointerException`, y la copia previa a `retainAll` garantiza tanto la no mutacion de `set1` como la nueva instancia de retorno.

3.10. Ejecucion

```
mvn -Dmaven.repo.local=.m2 -Dtest=IntersectionTest test
```

4. Ejercicio 4: PatternIndex – PWC y deteccion de fallas

Se modela y prueba `PatternIndex.patternIndex(String subject, String pattern)` con *Pair-Wise Coverage*, y se documentan las fallas que la suite revela en la implementacion original.

4.1. Archivos trabajados

- `src/main/java/assignment3_exercises/PatternIndex.java`
- `src/test/java/assignment3_exercises/PatternIndexTest.java`

4.2. Modelo del Dominio de Entradas

El MDE combina dos características de interfaz (referencia de cada parametro) con tres de funcionalidad (tamano y posicion de la ocurrencia esperada):

- **C1 – referencia de subject:** C1.b1 = null, C1.b2 = no nulo.
- **C2 – referencia de pattern:** C2.b1 = null, C2.b2 = no nulo.
- **C3 – tamano de subject** (si C1.b2): C3.b1 = vacio, C3.b2 = no vacio.
- **C4 – tamano de pattern** (si C2.b2): C4.b1 = vacio, C4.b2 = 1 caracter, C4.b3 = 2 o mas caracteres.
- **C5 – posicion del resultado** (si C1.b2 y C2.b2): C5.b1 = indice 0 (inicio), C5.b2 = intermedio, C5.b3 = final, C5.b4 = -1 (no encontrado).

Restricciones:

- Si `subject` o `pattern` es `null`: precondition violada, `IllegalArgumentException`; C3–C5 no aplican.
- Si `pattern` es vacío (C4.b1): por especificacion devuelve 0 (cubre C5.b1).
- Si `subject` es vacío (C3.b1) y `pattern` no lo es (C4.b2 o C4.b3): el resultado es -1 (cubre C5.b4).

4.3. Requisitos PWC y casos

Los 20 requisitos de cobertura por pares (R01–R20) modelan combinaciones validas entre las características. La suite implementa 15 casos:

TC	subject	pattern	Salida	Cubre
TC1	<code>null</code>	<code>null</code>	IAE	R01
TC2	<code>null</code>	<code>.a"</code>	IAE	R02
TC3	<code>.abc"</code>	<code>null</code>	IAE	R03
TC4			0	R04, R05, R11
TC5	<code>.abc"</code>		0	R04, R06, R12
TC6		<code>.a"</code>	-1	R04, R07, R15
TC7	<code>.abc"</code>	<code>.a"</code>	0	R04, R08, R13
TC8	<code>.abc"</code>	<code>"b"</code>	1	R04, R08, R14
TC9	<code>.abc"</code>	<code>ç"</code>	2	R04, R08, R16
TC10	<code>.abcde"</code>	<code>.ab"</code>	0	R04, R09, R17
TC11	<code>.abcde"</code>	<code>çd"</code>	2	R04, R09, R18
TC12	<code>.abcde"</code>	<code>"de"</code>	3	R04, R09, R19
TC13	<code>.abcde"</code>	<code>"fg"</code>	-1	R04, R09, R20
TC14	<code>.abc"</code>	<code>"z"</code>	-1	R04, R08, R15
TC15		<code>.ab"</code>	-1	R04, R10, R20

4.4. Fallas detectadas en la implementacion original

La version provista en el repositorio *no* sigue la especificacion documentada en el Javadoc. Los tests revelan dos defectos concretos:

1. **Ausencia de validacion de `null`.** La rutina invoca directamente `subject.length()` y `pattern.length()`, por lo que TC1–TC3 lanzan `NullPointerException` en lugar del `IllegalArgumentException` prometido.
2. **Tratamiento incorrecto de `pattern` vacío.** Con `pattern.length() == 0`, la condicion `iSub + patternLen - 1 < subjectLen` evalua `iSub - 1 < subjectLen`, lo cual es

verdadero al menos para $iSub = 0$. Dentro del cuerpo se ejecuta `pattern.charAt(0)`, que dispara `StringIndexOutOfBoundsException` en lugar de devolver 0.

4.5. Correccion aplicada

Se anaden las dos guardas faltantes al principio del metodo:

```
public static int patternIndex(String subject, String pattern) {
    if (subject == null || pattern == null) {
        throw new IllegalArgumentException("subject and pattern
            must be non-null");
    }

    final int NOTFOUND = -1;
    int iSub = 0, rtnIndex = NOTFOUND;
    boolean isPat = false;
    int subjectLen = subject.length();
    int patternLen = pattern.length();

    if (patternLen == 0) {
        return 0;
    }

    while (isPat == false && iSub + patternLen - 1 < subjectLen)
    {
        if (subject.charAt(iSub) == pattern.charAt(0)) {
            rtnIndex = iSub;
            isPat = true;
            for (int iPat = 1; iPat < patternLen; iPat++) {
                if (subject.charAt(iSub + iPat) != pattern.charAt
                    (iPat)) {
                    rtnIndex = NOTFOUND;
                    isPat = false;
                    break;
                }
            }
        }
        iSub++;
    }
    return rtnIndex;
}
```

Con estas dos correcciones, los 15 casos de la suite pasan: las precondiciones lanzan `IllegalArgumentException`, el patron vacio devuelve 0 y los demas casos no se ven afectados porque el algoritmo de busqueda original es correcto.

4.6. Ejecucion

```
mvn -Dmaven.repo.local=.m2 -Dtest=PatternIndexTest test
```

5. Ejercicio 5: Iterator sobre ArrayList – PWC

Se prueba la interfaz `java.util.Iterator` usando `ArrayList` como coleccion subyacente (Iterator es interfaz y no se instancia directamente).

5.1. Archivos trabajados

- `src/test/java/assignment3_exercises/IteratorArrayListTest.java`

5.2. Especificacion de referencia

De Oracle Java SE 7 (<https://docs.oracle.com/javase/7/docs/api/java/util/Iterator.html>):

- `hasNext()`: devuelve `true` si quedan elementos por iterar.
- `next()`: devuelve el proximo elemento; lanza `NoSuchElementException` si no hay mas.
- `remove()`: operacion opcional. Si no esta soportada, lanza `UnsupportedOperationException`. Si esta soportada, lanza `IllegalStateException` cuando no se llamo a `next()` previamente o cuando ya se llamo a `remove()` despues del ultimo `next()`.

5.3. MDE

- **C1**: “hay mas elementos” $\in \{\text{true}, \text{false}\}$.
- **C2**: “`next()` devuelve no nulo” $\in \{\text{true}, \text{false}\}$.
- **C3**: “`remove()` esta soportado” $\in \{\text{true}, \text{false}\}$.
- **C4**: “precondicion de `remove()` satisfecha” $\in \{\text{true}, \text{false}\}$.

Decisiones de instanciacion:

- Para `C3 = true` se usa `new ArrayList<>(...).iterator()`, que sí soporta `remove()`.

- Para `C3 = false` se usa `Collections.unmodifiableList(...).iterator()`, cuya implementacion delega en una vista que **no** soporta `remove()`, manteniendo un `ArrayList` como coleccion subyacente.

5.4. Requisitos y casos

Se cubren 13 requisitos:

TC	R	Escenario
TC1	R01	<code>hasNext()</code> = true sobre iterador con datos.
TC2	R02	<code>hasNext()</code> = false sobre iterador vacio.
TC3	R03	<code>hasNext()</code> = false sobre iterador agotado.
TC4	R04	<code>next()</code> devuelve un valor no nulo.
TC5	R05	<code>next()</code> devuelve <code>null</code> (la lista contiene un <code>null</code>).
TC6	R06	<code>next()</code> sobre iterador vacio → <code>NoSuchElementException</code> .
TC7	R07	<code>next()</code> sobre iterador agotado → <code>NoSuchElementException</code> .
TC8	R08	<code>remove()</code> no soportado (<code>unmodifiableList</code>) → UOE.
TC9	R09	<code>remove()</code> soportado pero sin <code>next()</code> previo → ISE.
TC10	R10	<code>remove()</code> sobre iterador vacio sin <code>next()</code> previo → ISE.
TC11	R11	<code>remove()</code> elimina el ultimo retornado con elementos restantes.
TC12	R12	<code>remove()</code> elimina el ultimo retornado sin elementos restantes.
TC13	R13	Doble <code>remove()</code> consecutivo → ISE.

5.5. Resultado

La implementacion de `Iterator` para `ArrayList` (y la vista de `unmodifiableList` para el caso `C3 = false`) respeta la especificacion en todos los escenarios modelados. No se detectaron divergencias respecto del contrato documentado.

5.6. Ejecucion

```
mvn -Dmaven.repo.local=.m2 -Dtest=IteratorArrayListTest test
```

Conclusiones

La resolución consolida el trabajo con *Input Space Partitioning* sobre cinco unidades concretas. Tres lecciones se repiten a lo largo del práctico:

1. **El MDE no es opcional.** En el Ejercicio 3 quedo claro que un MDE descuidado (C3 con bloques no disjuntos y no completos) hace inútil cualquier criterio que se aplique encima. Antes de generar requisitos hay que verificar completitud y disjointness.
2. **PWC vs BCC.** PWC fuerza interacciones; BCC fuerza variaciones controladas alrededor de un camino feliz. La elección depende del problema: PWC es más exhaustivo en interacciones (ejercicios 2 y 4), BCC concentra el esfuerzo en variaciones locales y se ajusta bien cuando se tiene un caso “normal” claro (ejercicio 3).
3. **Trazabilidad.** Anclar cada test a un TR explícito hace que la suite sea auditable y revisable: se sabe *por que* esta cada caso y que pasa si se elimina.